Sign your name:

IOAI 2025 GAITE Day 1

Team Leader's Translation Version

Team Leaders must complete the form below, even if they don’ t need translated documents.

Country or Region	Puerto Rico
Number of Team Members (sample papers will be duplicated according to the number)	4
How many pages in total (cover pages included)	24

Sign your name:

IOAI2025 GAITE: Radar

Note: Please "join" the competition first. Then, you can mount the dataset to the GPU. Otherwise, the notebook may encounter an error because it cannot access the dataset until you have joined the competition.

1. Problem Description

Radar is a key technology in wireless communication, with widespread applications such as self-driving cars. It typically involves an antenna that transmits specific signals and receives their reflections from objects in the environment. By processing these signals, the system determines the angular direction, distance, and velocity of target objects.

In real-world applications, radar signal processing is challenging due to noise and reflections from non-target objects in the surroundings. For example, when attempting to detect pedestrians, the radar may also receive reflections from trees or other background objects, which can degrade accuracy. Your task is to use AI to analyze the signals received by the radar and identify the presence of a human at each position.

In this task, we provide an **indoor radar experiment dataset**, and your objective is to develop a model that performs **radar semantic segmentation**.

2. Dataset

To measure objects surrounding a radar, the following key parameters are used:

- **Range**: The straight-line distance between the radar and an object.
- **Azimuth**: The horizontal angle (left to right) between the radar and the object.
- **Elevation**: The vertical angle (up or down) of the object relative to the radar.
- **Velocity**: The speed at which the object is moving toward or away from the radar.

The radar data is processed into multiple **heatmaps**, each encoding the **received signal strength** at various positions and directions.

- **Static heatmaps** emphasize reflections from **stationary** objects.
- **Dynamic heatmaps** highlight changes caused by **moving** objects.

When no object is present at a specific location, the signal consists mostly of background noise and appears weak. In contrast, reflections from an object increase signal intensity, enabling detection of the object.

For example, the **static range-azimuth heatmap** represents signal strength across different distances (**range**) and horizontal angles (**azimuth**), mainly reflected by stationary objects.

Each sample in the dataset is stored in a mat.pt file as a tensor of shape 7 x 50 x 181, where:

- 7 is the number of maps (6 heatmaps + 1 semantic label map),
- 50 represents range bins (distance),
- 181 represents angular or velocity bins, covering angles from -90° to +90° in either the horizontal or vertical plane. You can assume that the velocity bins are also remapped from -90° to +90° for visualization consistency.
- each heatmap intensity value is normalized to [0, 1], representing received signal strength.

The 6 heatmaps are structured as follows:

- **Index 0**: Static range-azimuth heatmap
- **Index 1**: Dynamic range-azimuth heatmap
- **Index 2**: Static range-elevation heatmap
- **Index 3**: Dynamic range-elevation heatmap
- **Index 4**: Static range-velocity heatmap
- **Index 5**: Dynamic range-velocity heatmap

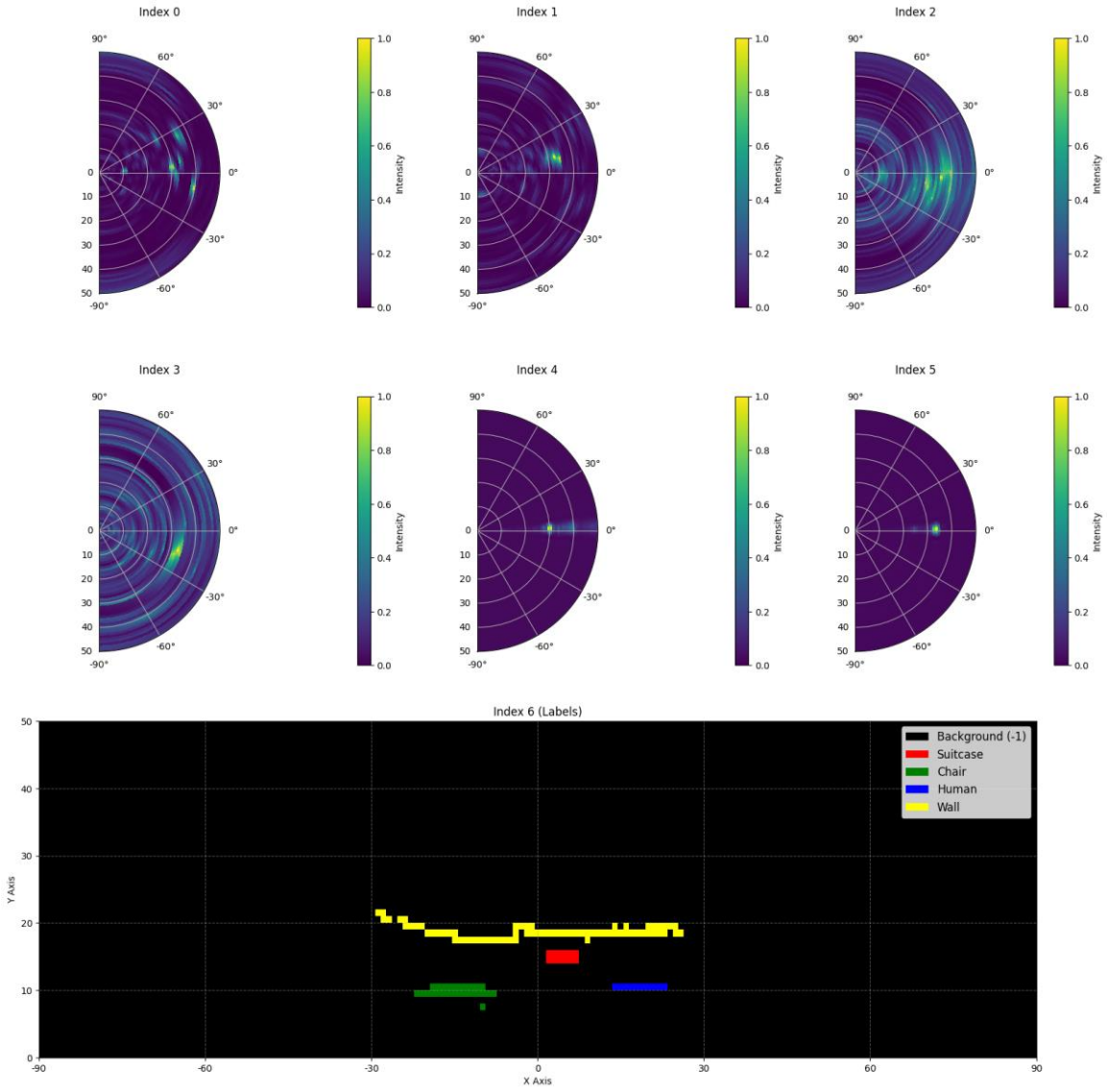
All values in heatmaps are **normalized**, so no unit conversion is required.

The **map at Index 6** is the semantic label map, stored in range-azimuth format.

- **-1**: Background (no target)
- **0**: Suitcase
- **1**: Chair
- **2**: Human
- **3**: Wall

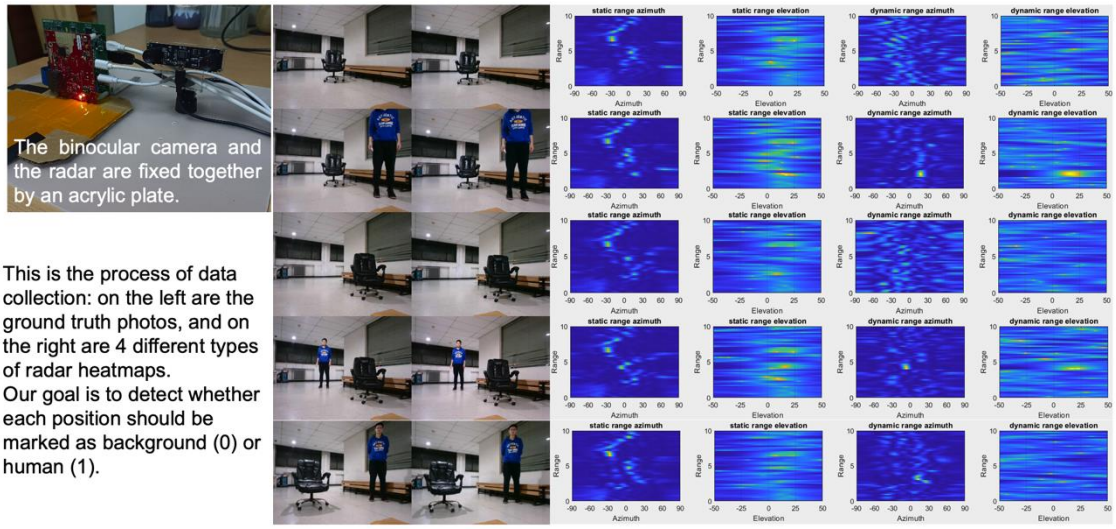
This is the visualization of 1 mat.pt in training_set:

Sign your name:



Here is part of a sample from the dataset:

Sign your name:



Ground Truth

Data scale: 1800 samples in the training set, 500 samples in the validation set, and 500 samples in the test set.

3. Task

Your task is to develop a model that takes the **first six heatmaps** (indices 0 to 5) as input, and predicts the **semantic label map** (index 6) as the output. The goal is to accurately identify what the target is (-1 to 3) at each location in the radar's field of view.

1. **Input**: A tensor of shape 6 x 50 x 181, representing six radar heatmaps.
2. **Output**: A tensor of shape 50 x 181, representing the target semantic label map.

4. Submission

Please submit a file named submission.ipynb. The output is a zip file named "submission.zip", which contains two tables submission_val.csv and submission_test.csv corresponding to the prediction results of the validation set and the test set respectively.

Note: The output table should have a header, the data in the table is not the actual solved data, it is only used as an example of the submission format.

filename	pixel_0	pixel_1	...	pixel_9049
1.mat.pt	-1	-1	...	-1
...	...	...	...	...

5. Score

The score is based on the **accuracy of label recognition**. Correctly identifying target points is weighted more heavily than correctly identifying background points.

Scoring Criteria:

- Each correctly identified **background pixel** earns **1 point**.
- Each correctly identified **non-background pixel** earns **50 points**.
- The final score is normalized to a **0-1 point** by comparing it to the maximum possible score.

Formula:

$$Score = \frac{\left|C_{0,correct}\right| \times 1 + \left|C_{1,correct}\right| \times bonus}{|C_0| \times 1 + |C_1| \times bonus}$$

where:

$$\begin{aligned} I &= \{1,2,\dots,50 \times 181\} \\ C_0 &= \{i \in I \mid y_i = -1\} \\ C_1 &= \{i \in I \mid y_i \neq -1\} \\ C_{0,correct} &= \{i \in C_0 \mid p_i = y_i\} \\ C_{1,correct} &= \{i \in C_1 \mid p_i = y_i\} \end{aligned}$$

Example

For a 3x3 heatmap, assume the Ground Truth is

$$\begin{bmatrix} -1 & -1 & -1 \\ 1 & 2 & 3 \\ -1 & -1 & -1 \end{bmatrix}$$

The result you identified is

$$\begin{bmatrix} -1 & 1 & -1 \\ -1 & 2 & -1 \\ -1 & 3 & -1 \end{bmatrix}$$

Then there are four correctly identified -1 and one correctly identified 2. Your score is 4 + 50 = 54 points. The maximum possible score is 6 + 50 * 3 = 156, that is, the score for six background pixels and three non-background pixels. Your normalized score is 54 / 156 = 0.346.

$$Score = \frac{4 \times 1 + 1 \times 50}{6 \times 1 + 3 \times 50} = 0.346$$

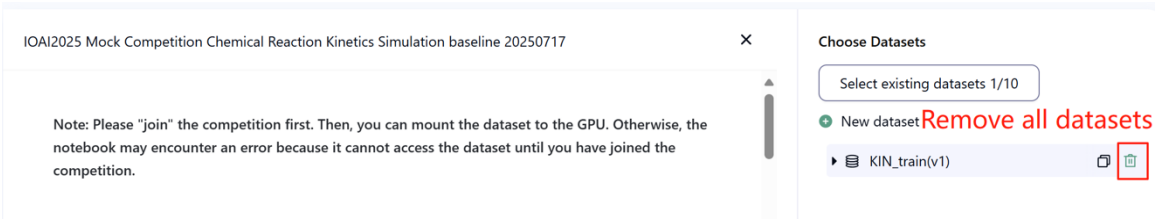
6. Baseline and Training Set

- The baseline is in **baseline.ipynb**.

- The dataset is in **training set**.

7. Requirements

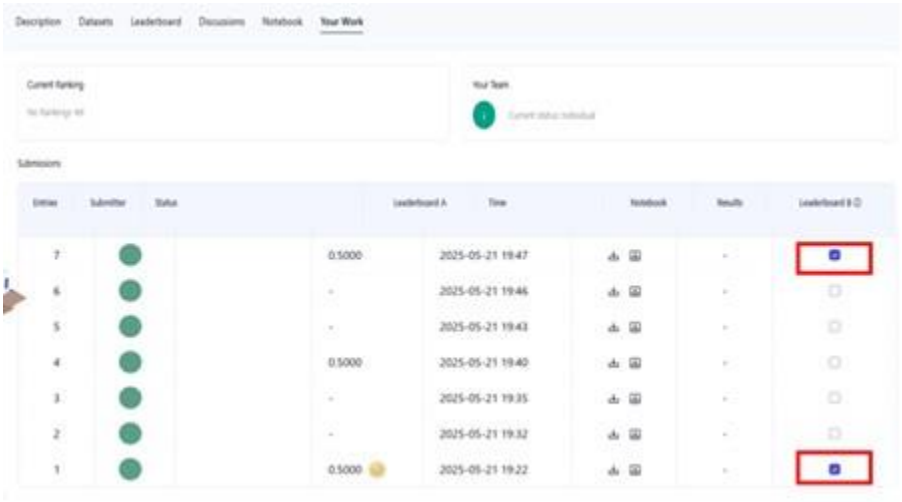
- Maximum submission limit: **15 times**. Only successful submissions (i.e., those receive a score on Leaderboard A) will be counted toward the submission limit.
- Testing environment restrictions: The test machine will run your Notebook within **15 minutes**. If the execution time exceeds **15 minutes**, the system will forcibly terminate and return a feedback of “Timeout” or “Failed”.
- Data and model submission: In this task, contestants can only submit a Notebook and **cannot** submit any mounted datasets or .pth files generated by themselves. Please check the upper right corner to remove the mounted dataset first.



- Network: For the on-site stage, the test machine cannot connect to the internet. In other words, downloading commands such as 'pip' and 'conda' or trying to call APIs will not work.
- Pretrained Model: Any pre-trained model can be used in this task when it can be imported properly without network connection and downloading. This message means the Bohrium cannot guarantee to exclude all the pretrained model in the Python image when install the packages, when you find some useful ones, you can use them.

8. Precautions

- Which score is effective: Contestants can select up to 2 submission results for scoring (√ - selected, □ - not selected). The score before unification for this task will be determined by the higher score on the Leaderboard B among the two selected submissions. Other cases of score calculation: please refer to **Appendix Platform Mechanisms and Restrictions for Individual Contest & GAITE**.



- How to deal with ambiguity: Once there is a conflict between the task description and the training set, the validation set and test set, the dataset will be respected first, and the dataset will not be changed during the competition.

- Contestants can only access Leaderboard A during the contest and cannot access Leaderboard B. The final score will be calculated only based on the score in Leaderboard B.

- The highest score by the Scientific Committee for this task is 0.90 in Leaderboard B, this score is used for score unification.

- The baseline score by the Scientific Committee for this task is 0.67 in Leaderboard B, this score is used for score unification.

9. Hints

To help participants with limited machine learning experience achieve good results, here are key strategies and general guidelines aligned with the task requirements:

(1). Understand the Data Structure

- **Input**: The input consists of 6 heatmaps (shape: 6×50×181) capturing radar signals from different perspectives: static/dynamic range-azimuth, range-elevation, and range-velocity. These heatmaps are normalized to [0, 1], so no further preprocessing for scaling is needed.

- **Output**: The target is a semantic label map (shape: 50×181) with 5 classes: -1 (background), 0 (suitcase), 1 (chair), 2 (human), 3 (wall).

- Focus on how each heatmap type (static vs. dynamic) encodes information: static maps highlight stationary objects, while dynamic maps emphasize moving ones.

(2). Model Design Principles

- **Multi-branch Architecture**: Consider designing separate feature extractors for each of the 6 heatmaps. This allows the model to learn unique patterns from range-azimuth, range-elevation, and range-velocity data independently before combining them.
- **Encoder-Decoder Structure**: Use encoders to compress high-dimensional heatmaps into compact features (via convolution layers) and decoders to upsample these features back to the original 50×181 size (via transposed convolution or upsampling). This helps preserve spatial details critical for segmentation.
- **Feature Fusion**: After processing each heatmap, merge their features effectively. Ensure alignment of spatial dimensions (e.g., elevation/velocity angles) to the target range-azimuth format before fusion. Simple concatenation or weighted summation of features can work well.

(3). Training Strategy

- **Loss Function**: Prioritize loss functions that handle class imbalance. Since non-background classes (0-3) are weighted more heavily in scoring, use methods like focal loss to focus training on hard-to-classify target pixels rather than overwhelming background pixels.
- **Optimizer**: Choose adaptive optimizers (e.g., Adam) for stable convergence, especially with high-dimensional radar features. Start with a moderate learning rate (e.g., 1e-3) and adjust based on validation performance.
- **Data Splitting**: Use the provided training set (1800 samples) for training and validation set (500 samples) to monitor overfitting. Avoid training on validation data to ensure reliable performance estimates.

(4). Handling Imbalanced Classes

- **Class Weights**: Explicitly assign higher weights to non-background classes in the loss function to match the scoring criteria (non-background correct predictions count 50x more).
- **Augmentation**: If possible, apply lightweight data augmentation (e.g., adding small noise to heatmaps) to increase diversity of training samples, especially for rare classes like "suitcase" or "human".

(5). Validation and Debugging

- **Track Key Metrics**: Monitor both overall loss and per-class accuracy, with extra attention to non-background classes. A model with high background accuracy but poor target detection will score low.
- **Visual Inspection**: Periodically plot sample predictions against ground truth to identify patterns (e.g., whether "human" is frequently confused with "chair") and adjust the model accordingly.

(6). Efficiency Considerations

- **Model Size**: Keep the model compact enough to run within the 15-minute time limit. Avoid excessive layers or large channel sizes unless justified by performance gains.
- **Batch Size**: Use a batch size that balances GPU memory usage and training stability (e.g., 4-8 for most setups).

By focusing on these areas—structured feature extraction, balanced training, and alignment with the scoring criteria—you can develop a robust model for radar semantic segmentation.

Sign your name:

IOAI2025 GAITE: Chicken Counting Problem

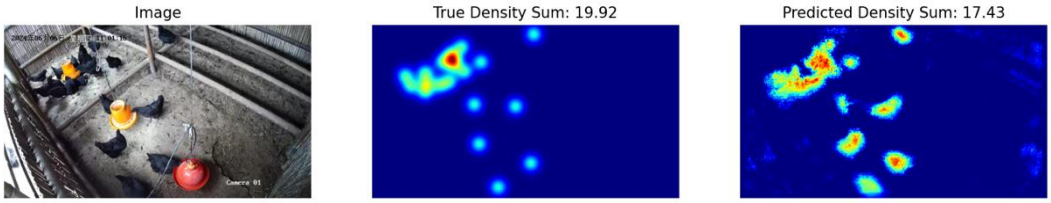
Note: Please "join" the competition first. Then, you can mount the dataset to the GPU. Otherwise, the notebook may encounter an error because it cannot access the dataset until you have joined the competition.

1. Problem Description

As the leader of an AI research team collaborating with Silkie chicken farmers, you are tasked with solving a critical challenge in traditional free-range farming. Accurate counting of livestock is crucial for both farmers and insurance companies, as factors like disease outbreaks and predator invasions can significantly impact the survival rate of these chickens in a short time. While insurance coverage helps mitigate farming risks, the claims process requires precise counting of livestock losses. Your farmers have approached your team for help in developing more accurate, automated counting systems. The challenge before your research team is to develop an optimized Silkie chicken counting model using density estimation techniques that can provide reliable counts to support both farm management and insurance processes.

Your team has access to a pretrained feature extractor for Silkie chicken images, but you'll need to design and train the density estimation decoder to create a complete counting solution. Your task is to build upon this foundation by developing an effective decoder architecture and training strategy to achieve accurate chicken counts that farmers and insurance companies can rely on.

The below figure shows an image in the dataset, as well as the corresponding true density distribution and a predicted density distribution generated by the baseline model. The total density (sum of densities across all areas) is labeled.



2. Dataset

The structure of the provided Silkie chicken image dataset is as follows:

```
datasets/
├── train/
│   └── A dataset with features:
│       ├── `image`: `PIL.Image` with RGB channels (3x720x1280)
│       └── `density`: a 2D array of shape 180x320
└── base.pth (Pretrained Model)
```

```
os.environ.get("DATA_PATH")/
├── test_a/
│   └── A dataset with features:
│       └── `image`: `PIL.Image` with RGB channels (3x720x1280)
└── test_b/
    └── A dataset with features:
        └── `image`: `PIL.Image` with RGB channels (3x720x1280)
```

(1) Training set location: datasets, files in this folder are used for model fine-tuning. It contains a train folder, which stores a dataset with 100 images and their corresponding density maps.

(2) Validation set (test_a) and Test set (test_b): These will be used to evaluate scores on Leaderboard A and Leaderboard B, respectively. They will be inaccessible to contestants. Only the score achieved on test set B will be used for final scoring.

(3) Datasets size:

- Training set: 100 images.
- Validation set: 100 images.
- Test set: 100 images.

(4) Validation set (test_a) and Test set (test_b) are not visible.

(5) Due to limits of computing resources, training density maps are reshaped to $1 \times 180 \times 320$. **NOTE** the output density map can be viewed as a 2D real number matrix with shape 180×320, the sum of all matrix values is the count of chickens.

3. Task

Your task is to train your own model using the training data to predict density maps, thereby serving the purpose of chicken counting.

You may extend and optimize the given pretrained model to improve its count prediction accuracy. The pretrained model base.pth only contains the weights of the first four layers of the model (the feature extraction model), and the function load_pretrained_weights_partial in the baseline code can be used to load these partial weights into your model. You may construct a density decoder DensityDecoder and combine it with the pretrained feature extraction module to form a complete data prediction model.

```
class DensityDecoder(nn.Module):
    def __init__(self):
        ################################################
        # Your code here
        ################################################

    def forward(self, x):
```

```
        ##################################################
        # Your code here
        ##################################################
        return x
```

You can also build your own model without the pretrained model we provided.

This task is the continuation of Satellite Weather Forecasting. A kind remind is the UNET is easily to full GPU memory without any feature engineering. Then, the GPU memory error message will be reported.

Please follow these rules to achieve a score normally:

(1) Your model must output the predicted density map.

(2) Due to limits of computing resources, your output density map should be reshaped to 180×320. This is also the shape of target density maps provided in the train dataset.

4. Submission

Please submit a **submission.ipynb** that includes the following components:

（1）**Training Code**

- Include the full training pipeline.

（2）**Evaluation Code**

- Evaluate your model on the validation set and test set. Refer to **baseline.ipynb** on how to access these datasets with a secret DATA_PATH.
- The output should be saved as **submission.npz**. This must be a valid npz file containing two arrays pred_a and pred_b, each with shape 100x1x180x320 (The evaluation script will also accept predictions in the shape of 100x180x320, if you decide to squeeze the channel dimension).

  Any result that does not meet the specified size will be considered invalid, resulting in an assessment score of zero.
- Each element of the density map should be **no less than zero**, otherwise will result in an assessment score of zero.

5. Score

You will be scored based on the mean relative error of your model. Relative error is defined by:

$$\text{Relative Error} = \frac{|y_i - \hat{y}_i|}{|y_i|}$$

where y_i is the true total density for the i-th sample, and $\hat{y}_i$ is the predicted total density for the i-th sample.

Your final score before normalization will be calculated based on your mean relative error, as follows:

$$\text{Score}=\exp\left(-\frac{1}{n}\sum_{i=1}^{n}\frac{|y_i-\hat{y}_i|}{y_i}\right)$$

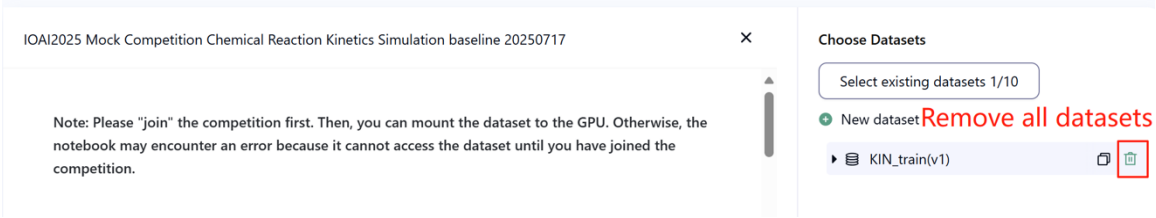
6. Baseline and Training Set

The baseline is in **baseline.ipynb**.

The training set is in **training set**.

7. Requirements

- Maximum submission limit: **15 times**. Only successful submissions (i.e., those receive a score on Leaderboard A) will be counted toward the submission limit.
- Testing environment restrictions: The test machine will run your Notebook within **15 minutes**. If the execution time exceeds **15 minutes**, the system will forcibly terminate and return a feedback of "Timeout" or "Failed".
- Data and model submission: In this task, contestants can only submit a Notebook and **cannot** submit any mounted datasets or .pth files generated by themselves. Please check the upright corner to remove the mounted dataset first.

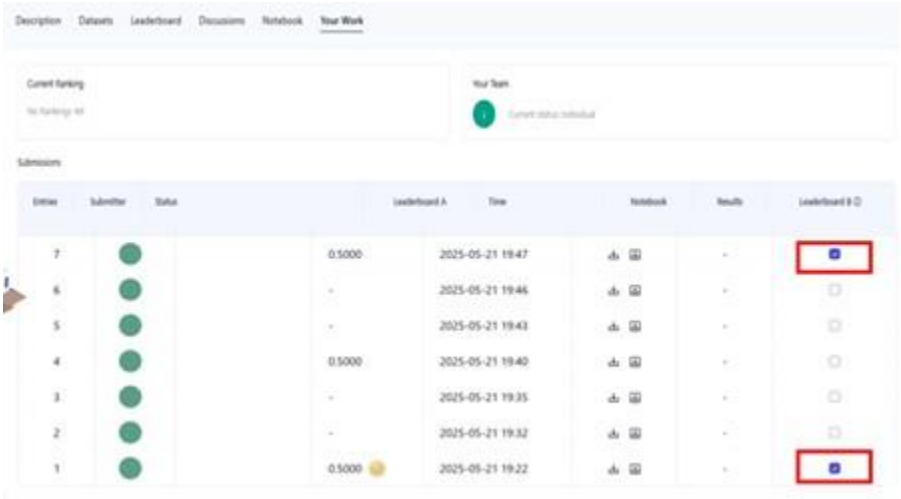


- Network: For the on-site stage, the test machine cannot connect to the internet. In other words, downloading commands such as 'pip' and 'conda' or trying to call APIs will not work.
- Pretrained Model: You may use the provided pretrained weights at **base.pth**, but it's not a must.

  Any other pretrained model can be used in this task when it can be imported properly without network connection and downloading. This message means the Bohrium cannot guarantee to exclude all the pretrained model in the Python image when install the packages, when you find some useful ones, you can use them.

8. Precautions

- Which score is effective: Contestants can select up to 2 submission results for scoring (√ - selected, □ - not selected). The score before unification for this task will be determined by the higher score on the Leaderboard B among the two selected submissions. Other cases of score calculation: please refer to **Appendix Platform Mechanisms and Restrictions for Individual Contest & GAITE**.



- How to deal with ambiguity: Once there is a conflict between the task description and the training set, the validation set and test set , the datasets will be respected first, and all the datasets will not be changed during the competition.

- Contestants can only access Leaderboard A during the contest and cannot access Leaderboard B. The final score will be calculated only based on the score in Leaderboard B.

- The highest score by the Scientific Committee for this task is 0.89, this score is used for score unification.

- The baseline score by the Scientific Committee for this task is 0.71, this score is used for score unification.

9. Hints

To assist participants in developing an effective chicken counting model using density estimation, here are key strategies and principles to guide your approach:

(1). Understand the Task and Data

- **Core Goal**: Predict a density map (shape: 180×320) where the sum of all values equals the number of chickens in the image. The final score depends on the mean relative error between predicted and true counts, so accuracy for both small and large flocks matters.

- **Data Characteristics**: The training set is small (100 images), so maximizing the use of available data and avoiding overfitting is critical. Density maps encode spatial distribution of chickens, with higher values indicating regions where chickens are present.

(2). Model Architecture Design

- **Feature Extraction + Density Decoding**: A common effective approach is to split the model into two parts:
    - A **feature extractor** that processes input images (3×720×1280) into compact, high-level features (e.g., using convolutional layers to capture patterns like chicken shapes, textures, or groupings).
    - A **density decoder** that transforms these features into the target 180×320 density map. Decoders often use upsampling (e.g., transposed convolution or bilinear upsampling) to recover spatial resolution, ensuring fine-grained density predictions.
- **Preserve Spatial Details**: Use architectures with skip connections (e.g., combining high-resolution early features with low-resolution deep features) to retain local information, which is vital for accurately locating individual chickens.

(3). Training Strategies

- **Loss Function Selection**: Prioritize loss functions that align with the scoring metric. Mean Absolute Error (MAE) is robust to outliers and helps minimize absolute differences between predicted and true density maps—critical since small absolute errors can lead to large relative errors for small flocks. Mean Squared Error (MSE) can also be used to penalize larger errors more heavily.
- **Handling Small Datasets**: To avoid overfitting with only 100 training images:
    - Use data augmentation (e.g., random rotations, mild scaling, brightness adjustments) to artificially expand the dataset.
    - Apply regularization (e.g., weight decay, dropout) to constrain model complexity.
- **Learning Rate and Scheduling**: Start with a moderate learning rate (e.g., 1e-4) and use a scheduler to reduce it gradually (e.g., step-wise decay). This stabilizes training and helps the model converge to better minima.

(4). Critical Constraints for Validity

- **Non-Negative Outputs**: Density maps must contain only non-negative values (since "negative chickens" are impossible). Ensure your model's final layer uses an activation like ReLU to enforce this—violations result in a score of zero.
- **Output Shape**: The predicted density map must strictly match the target shape (180×320). Verify the decoder's upsampling steps to ensure no dimension mismatches.

(5). Leveraging Pretrained Weights

- The provided base.pth contains pretrained weights for a feature extractor's early layers. Loading these weights can bootstrap your model with useful visual features (e.g., edges, textures) learned from larger datasets, reducing the need to train from scratch on the small training set. This is especially helpful for stabilizing training and improving feature quality.

(6). Efficiency Considerations

- **GPU Memory Management**: Avoid overly complex models (e.g., excessive convolutional channels) to prevent memory overflow, which can crash training or inference. Balance model depth and width to fit within the 15-minute runtime limit.
- **Inference Speed**: Ensure your model runs efficiently during evaluation, as slow inference may exceed the time limit for processing the validation and test sets.

(7). Focus on Relative Error

Since the score depends on relative error (not just absolute counts), pay attention to:

- Accuracy for small flocks (e.g., 1–5 chickens), where even a 1-chicken error can lead to a large relative error.
- Calibrating density map values to avoid systematic overcounting or undercounting (e.g., ensuring the sum of the density map aligns with true counts across different flock sizes).

By combining these strategies—careful architecture design, smart training practices, and adherence to output constraints—you can develop a robust density estimation model for accurate chicken counting.

Sign your name:

IOAI2025 GAITE: Resonance Elf (Straight Forward Problem)

Note: Please "join" the competition first. Then, you can mount the dataset to the GPU. Otherwise, the notebook may encounter an error because it cannot access the dataset until you have joined the competition.

1. Problem Description

On a distant planet, scientists are studying a mysterious creature. This creature is extremely sensitive to sound. When the sound frequency in the environment changes, it will exhibit different vitality values. Especially when the frequency of the environmental sound reaches a certain specific value, this creature will display astonishing vitality and energy. This characteristic reminds the scientists of the resonance phenomenon in physics, so they named this kind of creature Resonance Elf. The scientists believe that by understanding how the Resonance Sprite reacts to different sound frequencies, they can uncover the physiological structure of this creature and may even develop a new energy technology.

2. Dataset

In order to study the behavior of the Resonance Sprite, they defined an observable physical quantity, the amplitude A of the vitality value of the Resonance Sprite, and designed an experimental method that can precisely control the frequency ω of the sound. In a series of carefully designed experiments, they recorded the vitality values of three different Resonance Sprites at different sound frequencies, and stored the data in the following datasets respectively:

(1) Training set: data_training.csv, the address of the training set: to be filled in by the platform;

(2) Validation set: data_validation.csv. During the competition, contestants cannot directly download validation set;

(3) Testing set: data_testing.csv. During the competition, contestants cannot directly download testing set.

The first column of the data represents the frequency ω of the sound, denoted as omega; the second column represents the amplitude A of the vitality value, denoted as amp, as shown in the following table:

omega	amp
1.5	0.6
1.6	0.9
1.7	1.4
1.8	1.3
1.9	0.7
...	...

For the convenience of notation, all physical quantities are dimensionless, and contestants only need to operate on pure numbers.

Note: The training set data is fully visible, which can help contestants develop methods for solving parameters. The data of validation set is not visible to contestants, but it will be displayed on the Leaderboard A, which can help verify whether the method for solving parameters is correct. The data of testing is not visible to contestants and will be used to calculate the final scores on the Leaderboard B. Since the training set, validation set, and testing set come from three different Resonance Elves, their parameters are different.

3. Task

Scientists modeled the behavior of this creature by drawing on the resonance phenomenon in physics, and guessed that the functional relationship between the amplitude A of the vitality value and the frequency ω of the external sound is

$$A(\omega) = \frac{F_0}{m\sqrt{(\omega^2 - \omega_0^2)^2 + 4\delta^2\omega^2}}$$

where F_0 and m represent the amplitude of the external sound and the "effective mass" of the creature's sound response system respectively. During the calculation process, F_0 and m are fixed as 1. Scientists are more concerned about the two parameters, the resonance frequency ω_0 and the damping coefficient δ. It is not difficult to see that when the value of the sound frequency ω equals resonance frequency ω_0 , the amplitude A reaches the maximum value, which is the resonance phenomenon. The appearance of the damping coefficient δ ensures that the amplitude will not reach infinity during resonance. Therefore, scientists believe that the resonance and the damping coefficient actually characterize the biological characteristics of this sprite, and **the task objective is to obtain the resonance frequency ω_0 and the damping coefficient δ from the dataset (that is, the data composed of the amplitude A of the vitality value and the frequency ω of the external sound) for further research**.

4. Submission

Please submit the submission.ipynb file, which contains the entire process of training the model and solving the parameters.

The submission.ipynb should be able to generate three files:

3. The parameter-solving results for the training set are stored in submission_training.csv;
4. The parameter-solving results for the validation set are stored in submission_validation.csv;
5. The parameter-solving results for the testing set are stored in submission_testing.csv.

The format for storing parameters is shown in the table below, where w0_pred represents the predicted value of ω_0 and delta_pred represents the predicted value of δ. If some parameters are not solved, please fill in a default parameter, such as 1, and do not leave blanks.

w0_pred	delta_pred
1	1

The above three csv files should be packaged into a zip file according to their respective names. You can refer to the submission format in the **baseline.ipynb**.

5. Score

1. The final scoring matrix is as follows: where O_{pre} is the contestant's predicted value for parameter O, and O_{real} is the true value of parameter O;

   (1) Scoring for the solution of the resonance frequency ω_0 : $S_1 = \exp\left(-\left|\omega_{0\ \text{pre}} - \omega_{0\ \text{real}}\right| / \left|\omega_{0\ \text{real}}\right|\right)$

   (2) Scoring for the solution of the damping coefficient δ: $S_2 = \exp\left(-\left|\delta_{0\ \text{pre}} - \delta_{0\ \text{real}}\right| / \left|\delta_{0\ \text{real}}\right|\right)$

   (3) Final score: $\text{Score}=(S_1 + S_2)/2$.

2. Failure to submit in the required format: 0 points.

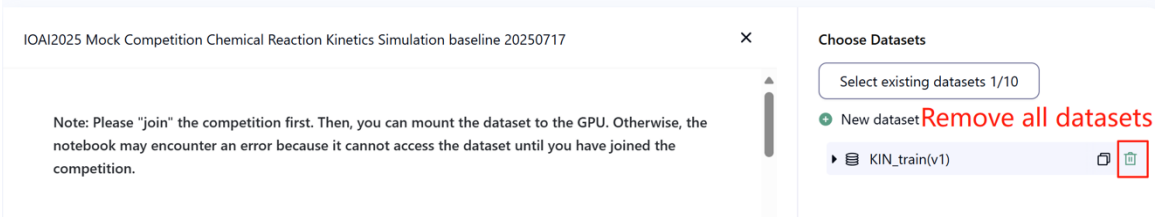
6. Baseline and Training Set

- The baseline is in **baseline.ipynb**.
- The training set is in **training set**.

7. Requirements

- Maximum submission limit: **15 times**. Only successful submissions (i.e., those receive a score on Leaderboard A) will be counted toward the submission limit.
- Testing environment restrictions: The test machine will run your Notebook within **15 minutes**. If the execution time exceeds **15 minutes**, the system will forcibly terminate and return a feedback of “timeout” or “failure”.
- Data and model submission: In this task, participants can only submit a Notebook and **cannot** submit any mounted datasets or .pth files generated by themselves. Please check the upright corner to remove the mounted dataset first.

Sign your name:

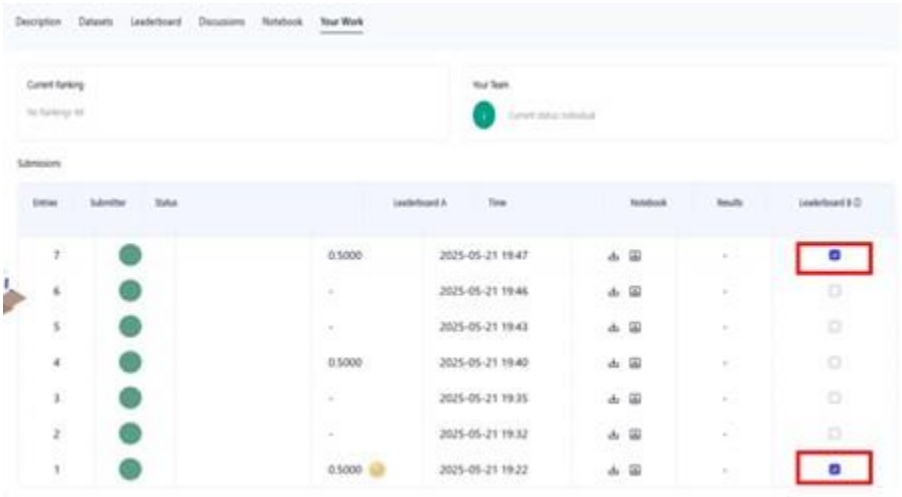


- Network: For the on-site stage, the test machine cannot connect to the internet. In other words, downloading commands such as 'pip' and 'conda' or trying to call APIs will not work.

- Pretrained Model: Any pre-trained model can be used in this task when it can be imported properly without network connection and downloading. This message means the Bohrium cannot guarantee to exclude all the pretrained model in the Python image when install the packages, when you find some useful ones, you can use them.

8. Precautions

- Which score is effective: Contestants can select up to 2 submission results for scoring (√ - selected, □ - not selected). The score before unification for this task will be determined by the higher score on the Leaderboard B among the two selected submissions. Other cases of score calculation: please refer to **Appendix Platform Mechanisms and Restrictions for Individual Contest & GAITE**.



- How to deal with ambiguity: Once there is a conflict between the task description and the training set, the validation set and test set , the dataset will be respected first, and the dataset will not be changed during the competition.

- Contestants can only access Leaderboard A during the contest and cannot access Leaderboard B. The final score will be calculated only based on the score in Leaderboard B.

- The highest score by the Scientific Committee for this task is is 0.99 in Leaderboard B, this score is used for score unification.
- The baseline score by the Scientific Committee for this task is 0.25 in Leaderboard B, this score is used for score unification.

9. Hints

To effectively determine the resonance parameters (ω_0 and δ) for the Resonance Elf dataset, consider these key strategies:

(1). Understand the Resonance Model

The relationship between amplitude (A) and frequency (ω) is defined by the physical model:

$$A(\omega) = \frac{F_0}{m\sqrt{\left(\omega^2 - \omega_0^2\right)^2 + 4\delta^2\omega^2}}$$

With F_0 and m fixed at 1, your task reduces to estimating two parameters:

- ω_0 (resonance frequency): The frequency where amplitude peaks (A is maximized).
- δ (damping coefficient): Controls the width of the resonance peak (smaller δ = sharper peak).

(2). Parameter Estimation Approach

- **Curve Fitting**: The core challenge is to find ω_0 and δ that best fit the observed (ω, A) data points. This is a nonlinear optimization problem where you minimize the difference between predicted and actual amplitudes.
- **Objective Function**: Use a loss function (e.g., mean squared error, MSE) to quantify the mismatch between the model's predicted amplitude and the observed amplitude from the data. Minimizing this loss yields optimal parameters.

(3). Optimization Techniques

- **Gradient Descent**: Efficient for this problem—iteratively adjust ω_0 and δ to reduce the loss. Start with reasonable initial guesses (e.g., ω_0 near the frequency of maximum observed amplitude; δ as a small positive value like 0.1).
- **Learning Rate**: Choose a moderate learning rate (e.g., 0.1–0.5) to balance convergence speed and stability. Too large a rate may cause overshooting; too small may slow convergence.
- **Training Epochs**: Use sufficient iterations (e.g., 10,000–20,000) to ensure the model converges to a stable minimum. Monitor the loss to confirm it plateaus, indicating convergence.

(4). Validation and Intuition

- **Visual Inspection**: Plot the observed data points alongside the model's predictions using the estimated parameters. A good fit should show the predicted curve closely following the data, with the peak aligned at the correct resonance frequency.

- **Resonance Peak Clues**: The observed amplitude will be highest near ω_0. Use this to refine initial guesses—start ω_0 near the ω value with the largest A in the dataset.

(5). Handling Different Datasets

Training, validation, and test sets correspond to different Resonance Elves with unique ω_0 and δ. The same fitting approach applies to all, but ensure your code dynamically loads each dataset and runs the optimization independently for each.

(6). Practical Implementation Tips

- **Numerical Stability**: Ensure δ remains positive (physically meaningful) during optimization (e.g., use a small lower bound or apply exponential activation to enforce positivity).

- **Efficiency**: The problem is computationally lightweight, so even simple optimizers will work within the 15-minute limit. Focus on ensuring convergence rather than complex algorithms.

By framing the problem as a nonlinear curve-fitting task and using gradient-based optimization to minimize the mismatch between model predictions and data, you can accurately estimate the resonance parameters.

Sign your name: _____

<< FINAL PAGE >>

<< FINAL PAGE >>